



Supporting signed char and unsigned char in random number generation

Document number:

[P4037R0](#)

Date:

2026-03-06

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:

Jan Schultke <janschultke@gmail.com>

GitHub Issue:

wg21.link/P4037/github

Source:

github.com/eisenwave/cpp-proposals/blob/master/src/random-chars.cow



ABSTRACT: Use of `signed char` or `unsigned char` in `<random>` currently results in undefined behavior. Instead, it should be permitted. There are further minor defects resulting from the restrictions in [\[rand.req.genl\]](#), which this paper also addresses. NB comment [\[RU-272\]](#) is resolved by this paper.

§ Contents

- 1 Introduction
- 2 Motivation
- 3 Design
 - 3.1 Which types to support
 - 3.1.1 Proposed `IntType` restrictions
 - 3.1.2 Proposed `RealType` restrictions
 - 3.2 Don't require character types
 - 3.3 Don't require all extended integer types
 - 3.4 Drive-by-fix for `std::uint_leastN_t` and `std::uint_fastN_t`
 - 3.5 Treating engines and distributions differently

4	Implementation experience
5	Wording
6	References



§ 1. Introduction



BUG: The following code has compile-time undefined behavior:

```
void generate_random_bytes(std::span<unsigned char> out) {
    std::default_random_engine rng(12345);
    std::uniform_int_distribution<unsigned char> distr; // UB
    for (unsigned char& c : out) {
        c = distr(rng);
    }
}
```

While it may seem perfectly reasonable to generate random bytes in the form of `unsigned char`, [\[rand.req.genl\]](#) requires that where `IntType` appears as a template parameter name (such as in `uniform_int_distribution`), providing `unsigned char` as an argument causes template instantiation to have undefined behavior. This is functionally equivalent to making the program ill-formed, no diagnostic required.

It should also be noted that many users rely on support for 8-bit types in `<random>` already, despite the undefined behavior. A GitHub code search for `language:C++ /(?-i)uniform_int_distribution<((unsigned|signed|)char|(std::)?u?int8_t|i8|u8)>/` shows 8.4K files already using e.g. `std::uniform_int_distribution<uint8_t>`. This is possible because both `libstdc++` and `libc++` support those types as an extension; see [§4. Implementation experience](#).

In 2013, [\[LWG2326\]](#) “`uniform_int_distribution<unsigned char>` should be permitted” stated that “it’s just silly that we have a random number library with no natural way to generate random bytes”. As silly as it may be, the issue was closed as NAD (not a defect) because it was perceived as a feature request rather than the resolution of a defect. Writing a paper was considered, but that paper never manifested.

In 2025, [\[LWG4109\]](#) “Instantiating templates in `§[rand]` with `int8_t/uint8_t` is undefined behavior” addressed the same problem, and addresses NB comment [\[RU-272\]](#). Various comments on [\[LWG4109\]](#) point out that support for extended integer types and extended floating-point types would be useful, and that many of the restrictions could be converted from compile-time UB (effectively IFNDR) to *Mandates*, i.e. `static_assert`.

This paper resolves all these known defects and discrepancies of [\[rand.req.genl\]](#) in one fell swoop.



NOTE: It is unclear why the prohibition of `unsigned char` exists in the first place. The restriction dates back at least to [\[N1932\]](#), published 2006. [Discussion on \[std-proposals\]](#) suggests that it may exist because engines with 8-bit types (like `linear_congruential_engine<uint8_t>`) have dreadful statistical properties.

§ 2. Motivation



Support for `unsigned char` and `signed char` in distributions would be useful. The ability to generate random bytes or octets is extremely valuable for [fuzz testing](#). For example, when testing an implementation of an [LZ77 or LZ78](#) compression algorithm, a user would typically

1. generate a random sequence of bytes,
2. compress that sequence using their algorithm,
3. decompress the result, and
4. compare whether the decompressed bytes are the same as the original bytes.

It is easy to imagine testing methodology that involves random bytes for any kind of codec, network protocol, compiler, syntax highlighter, etc.



NOTE: Generating octets is effectively supported already. `uniform_int_distribution<unsigned int>(0, 255)` is functionally equivalent to `uniform_int_distribution<unsigned char>()`.

§ 3. Design

§ 3.1. Which types to support

The current restriction which forbids `unsigned char` is bad, but it's not obvious how much the restriction should be relaxed.

While [\[LWG4109\]](#) offers the option to make the program ill-formed when the existing restrictions are not satisfied (“Option B” in the issue), it fails to mention that this would break thousands of files of existing code; see [§1. Introduction](#). We have to permit implementations to keep their current extensions, without requiring them to support more than is reasonable. We either have to continue treating anything other than the supported set of types as IFNDR (“Option A” in the issue), or carve out an optionally supported set of types (proposed).

§ 3.1.1. Proposed `IntType` restrictions

In short, the new restriction for `IntType` parameters is to ensure support for standard integers (including `signed char` and `unsigned char`), allow the implementation to support further integral types (such as extended integer types), and reject anything that is not even an integral type. However, some extended integer types are also required to be supported to prevent highly defective behavior. The following sections provide more details, and the following diagram illustrates the behavior:

REQUIRED: standard ints., some extended ints.
e.g. `unsigned char`, `int`, `int32_t`, etc.

OPTIONAL: cv-unqualified integral types
e.g. `__int128`, `char32_t`, etc.



```
REJECTED: anything else
e.g. std::vector, double, enum, const int, etc.
```

The `UIntType` restrictions are the same, except that signed types fall into the “anything else” set.

§ 3.1.2. Proposed `RealType` restrictions

The same restriction is mirrored for `RealType` parameters: there is mandatory support for `float`, `double`, and `long double`, and optional support extended floating-point types.

```
REQUIRED: cv-unqualified standard floating-point
float, double, and long double
```

```
OPTIONAL: cv-unqualified extended floating-point
e.g. float32_t, float128_t, bfloat16_t, etc.
```

```
REJECTED: anything else
e.g. std::vector, int, enum, const double, etc.
```

§ 3.2. Don't require character types

Supporting use of character types (e.g. `char`, `char32_t`, etc.) is not motivated by this paper and outside the scope of [RU-272]. It is not proposed, despite character types being integral types, despite seeming vaguely useful, and despite GCC having support for this already (see §4. [Implementation experience](#)). Such an extension could be proposed in a future paper targeting C++29.

Supporting only `char` in addition to standard integer would also be strange, although there is precedent for this in `<charconv>`. If `char` is supported, why not character types in general? The design becomes somewhat arbitrary.



NOTE: It is not permitted to define `std::int8_t` or `std::uint8_t` as `char`, even if `char` has the correct signedness.

§ 3.3. Don't require all extended integer types

Supporting extended integer types in general imposes an unreasonable implementation burden. One of the biggest issues is `linear_congruential_engine<__int128>` assuming the implementation provides an extended 128-bit integer type, which recent versions of GCC and Clang do. `libstdc++` permits `__int128` in most of `<random>`, but rejects `linear_congruential_engine<__int128>` using:



```
static_assert(__which < 0, /* needs to be dependent */
              "sorry, would be too much trouble for a slow result");
```



The difficulty lies in the fact that `linear_congruential_engine` uses modular arithmetic, which requires $N \times 2$ -bit division for N -bit integer types. For `long long`, the existence of `__int128` makes this easy, but for `__int128`, 256-bit division would be required.

However, *some* extended integer types ought to be supported; more on that in the following section.

§ 3.4. Drive-by-fix for `std::uint_leastN_t` and `std::uint_fastN_t`

Since we are changing the set of types that `IntType` and `UIntType` support, we can also fix another defect as a drive-by product: `[rand]` uses the type aliases `uint_least32_t`, `uint_least64_t`, `code{uint_fast32_t}`, and `uint_fast64_t` in a number of places, but there is no guarantee that those are aliases for standard integer types rather than extended integer types.



EXAMPLE: It is implementation-defined whether the following specialization from `[rand.predef]` causes compile-time undefined behavior upon instantiation. Specifically, there would be UB if `uint_fast32_t` was an alias for some `__uint32` extended integer type.

```
using mt19937 =  
    mersenne_twister_engine<uint_fast32_t, 32, 624, 397, 31,  
        0x9908'b0df, 11, 0xffff'ffff, 7, 0x9d2c'5680, 15, 0xefc6'0000, 18, 1'83
```

While it is obvious that we don't want the use of `mt19937` to have undefined behavior, it is not obvious how to fix the problem. Allowing *all* extended integer types would include `__int128`, which we don't want to support, as explained above.

A good compromise is to support all integer types whose width at least that of `char` and at most that of `long long`. This ensures that `std::size_t`, `std::int_least32_t`, and many other types are supported by `<random>`, regardless of whether they are standard integer types, assuming they don't have an extreme width (which the user can easily test using `static_assert`). This compromise is also future-proof for bit-precise integers (see [\[P3666R3\]](#)), which could introduce extreme widths like `_BitInt(1)` or `_BitInt(1024)` that we may not want to support.

§ 3.5. Treating engines and distributions differently

While the paper is motivated by supporting distributions, engines such as `mersenne_twister_engine` also use `UIntType` template parameters and thus disallow `unsigned char`. Using 8-bit types for engines is mostly unmotivated because the quality of the generated numbers would be extremely low anyway. That being said, using an 8-bit `linear_congruential_engine` to get “quick and dirty” random number generation on an 8-bit architecture is not inconceivable, and [GitHub code search](#) finds uses of `linear_congruential_engine<uint8_t>`.

I argue that `unsigned char` and `signed char` should be supported unilaterally in `<random>` (not just for distributions) for the following reasons:



- There are existing uses of 8-bit engine types, and we don't need to break this existing code, however questionable that existing code may be.
- It is unlikely that support for 8-bit types would cause ordinary users to *accidentally* create an engine of low quality. Ordinary users don't create their own engines with custom parameters; they typically reach for `std::mt19937` and other predefined aliases.
- Creating an engine of extremely low quality is useful for statistical libraries that measure said quality. Sometimes, low quality is the goal, not a mistake.
- A user can create a low-quality engine using both `linear_congruential_engine<uint8_t, 2u, 3u, 8u>` and `linear_congruential_engine<uint64_t, 2u, 3u, 8u>`. The statistical quality is determined by the engine parameters, not by the type.
- None of the other numerical templates in `<numeric>` or `<cmath>` single out a specific integer width as unsupported. This design is very surprising and actively hostile to generic programming.



NOTE: In terms of wording, supporting 8-bit types unilaterally means that `UIntType` template type parameters accept `unsigned char`.

§ 4. Implementation experience

At the time of writing, the MSVC STL has a `static_assert` which prevents instantiation of `std::uniform_int_distribution<unsigned char>`. It provides the minimum amount of support required by the standard; see:

<https://github.com/microsoft/STL/blob/9b021cf66875d2fbf4627aad8db594595aebb148/stl/inc/random#L43>

libc++ supports `signed char` and `unsigned char` as an extension. This is the minimum amount of support assuming this paper is accepted; see:

https://github.com/llvm/llvm-project/blob/9d075d271ff15ec153ada3413d294540206a15ed/libcxx/include/__random/is_valid.h#L49.

libstdc++ supports any integral type as an extension; see:

https://github.com/gcc-mirror/gcc/blob/0635bfb53a145cc005a15657635abf8ea9e6e9ba/libstdc%2B%2B-v3/include/bits/uniform_int_dist.h#L90-L91.

§ 5. Wording

The changes are relative to [N5032].



DRAFTING NOTE: No feature-test macro is provided because these changes are intended to be applied as a defect report.



DRAFTING NOTE: Remember that the following wording changes are meant to resolve three defects at once:

- `signed char` and `unsigned char` are not supported.
- `uint_least32_t` and other such types are not guaranteed to be supported.

- The effect of instantiating templates is undefined, but many of the requirements (e.g. cv-unqualified types) are diagnosable; even when they aren't, the wording idiom “ill-formed, no diagnostic required” is more fitting.



Change [\[rand.req.genl\]](#) paragraph 1 as follows:



Throughout [\[rand\]](#), the effect of instantiating a template *T* has the following constraints:

- that The program is ill-formed if *T* has a template type parameter named Sseq is undefined-unless and the corresponding template argument is cv-unqualified and-meets or does not meet the requirements of seed sequence ([\[rand.req.seedseq\]](#)); no diagnostic is required for the latter condition.
- that The program is ill-formed if *T* has a template type parameter named URBG is undefined-unless and the corresponding template argument is cv-unqualified and-meets or does not meet the requirements of uniform random bit generator ([\[rand.req.urng\]](#)); no diagnostic is required for the latter condition.
- that The program is ill-formed if *T* has a template type parameter named Engine is undefined-unless and the corresponding template argument is cv-unqualified and-meets or does not meet the requirements of random number engine ([\[rand.req.eng\]](#)); no diagnostic is required for the latter condition.
- that The program is ill-formed if *T* has a template type parameter named RealType is undefined-unless and the corresponding template argument *A* is cv-unqualified and is one of float, double, or long double is not a cv-unqualified floating-point type ([\[basic.fundamental\]](#)). It is implementation-defined whether the program is ill-formed if *A* is an extended floating-point type.
- that The program is ill-formed if *T* has a template type parameter named IntType is undefined-unless and the corresponding template argument is cv-unqualified and is one of short, int, long, long long, unsigned short, unsigned int, unsigned long, or unsigned long long *A* is not a cv-unqualified integral type ([\[basic.fundamental\]](#)). It is implementation-defined whether the program is ill-formed if *A* is not a signed or unsigned integer type or if the width of *A* is less than that of char or more than that of long long.
- that The program is ill-formed if *T* has a template type parameter named UIntType is undefined-unless and the corresponding template argument is cv-unqualified and is one of short, int, long, long long, unsigned short, unsigned int, unsigned long, or unsigned long long *A* is not a cv-unqualified integral type ([\[basic.fundamental\]](#)) or if is_signed_v<*A*> is ill-formed or true. It is implementation-defined whether the program is ill-formed if *A* is not an unsigned integer type or if the width of *A* is less than that of char or more than that of long long.

§ 6. References

[N1932] *Walter E. Brown et al.*. Random Number Generation in C++0X: A Comprehensive Proposal (2006-02-23)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2006/n1932.pdf>

[N5032] *Thomas Köppe*. Working Draft, Programming Languages — C++ (2025-12-15)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5032.pdf>

[LWG2326] *Stephan T. Lavavej*. `uniform_int_distribution<unsigned char>` should be permitted (2017-02-02)

<https://cplusplus.github.io/LWG/issue2326>

[LWG4109] *Peter Dimov*. Instantiating templates in `§[rand]` with `int8_t/uint8_t` is undefined behavior (2025-10-27)

<https://cplusplus.github.io/LWG/issue4109>

[RU-272] [rand.req.genl] Prevent instantiation of `uniform_int_distribution` with `std::uint8_t`

<https://github.com/cplusplus/nbballot/issues/847>

[P3666R3] *Jan Schultke*. Bit-precise integers (2026-02-21)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2026/p3666r3.pdf>

